SENG401 – GRADUATION PROJECT

Final Report

By Sinem Coşkun, Al-baraa Abdullah Abdullah Abdullah Al-yadumi

1. Introduction 3
2. Literature Review 3
2.1 IoT Security Ecosystem and Current Threats 3
2.2 Honeypot Technologies 3
2.3 Protocol Tunnelling and Data Exfiltration 4
2.4 The Importance of Feature Engineering 4
3. Methodology 4
3.1 Hardware Selection and Raspberry Pi Configuration 4
3.2. Network Architecture: Isolated Hybrid Monitoring Environment (Double NAT) 4
3.3 Honeypot Services and System Hardening 5
3.4 Pipeline and Central Analysis Software 5
3.4.1 Edge Layer Data Processing Logic 5
3.4.2 Communication Protocol: TLS Encrypted MQTT 6
3.4.3 Central Side (Host) and Data Recording 6
3.5 Feature Engineering 6
3.5.1 Statistical and Entropy-Based Features 6
3.5.2 Behavioral and Temporal Characteristics 6
3.5.3 Semantic and Intent Analysis Features 7
3.5.4 Threat Classification and Risk Scoring 7
4. Conclusion 7
5. Appendix 8
6. References 10

1. Introduction

The Internet of Things (IoT) ecosystem has a heterogeneous structure consisting of sensors, embedded systems, and lightweight communication protocols. The limited hardware resources (processing power, memory capacity, etc.) of these devices and the difficulties in software update processes create a large attack surface for cyber attackers. Analysis of 2025 data shows that a large proportion of IoT devices are still vulnerable to brute-force attacks, weak protocols, and firmware vulnerabilities.

The primary goal of this project is to establish a Raspberry Pi-based honeypot system capable of collecting real-time attack data, detecting tunneling activity, distinguishing between bot and human behavior, and identifying completed attacks. This work includes:

- Creating a deception environment for SSH, FTP, HTTP, and DNS protocols using Cowrie, Honeytrap, and Suricata software.
- The MQTT protocol was chosen for cost-effective and reliable transmission of the collected data.
- The raw log data obtained was stored in an SQLite-based database on the host system and subjected to Feature Engineering processes to provide meaningful analysis.

This phase of the report covers the data collection and feature extraction processes that will form the basis for statistical analysis and machine learning models. The project aims to contribute to the literature by creating a local and up-to-date attack dataset.

2. Literature Review

In cybersecurity literature, protecting IoT devices requires approaches beyond traditional security solutions due to the heterogeneous nature of these devices and their limited resources. This section examines honeypot technologies, protocol tunneling, and current threat approaches, which form the basis of this project, in light of academic studies.

2.1 IoT Security Ecosystem and Current Threats

Current analyses for 2025 show that a large portion of IoT devices are still the primary target of cyber attackers due to brute-force attacks and weak authentication protocols [1]. In particular, the use of embedded systems in the defense industry and critical infrastructure has made the security of these devices a national security issue [7][8]. Although distributed defense methods such as federated learning are suggested in the literature, it is emphasized that these models need to be constantly fed with real-time attack traffic [2].

2.2 Honeypot Technologies

Honeypots are decoy systems designed to deceive attackers and monitor their activities. In IoT security, honeypot data is considered the most valuable data source, especially for training machine learning-based intrusion detection systems (IDS) [3][4].

- ***Medium Interaction Honeypots:*** Systems like Cowrie emulate attacks made over SSH and Telnet, recording the attacker's command-line interactions.

- ***Service Emulation:*** Tools like Honeytrap mimic TCP/UDP-based services, providing a broad attack surface and capturing reconnaissance activities.

2.3 Protocol Tunnelling and Data Exfiltration

Protocol tunneling is the process of bypassing firewalls and network monitoring tools by hiding data within a carrier protocol (DNS, HTTP, ICMP, etc.).

- ***DNS Tunneling:*** The literature states that attackers frequently use this protocol for data leakage and command and control (C2) communication because DNS queries are often passed unchecked [10].
- ***Detection Methods:*** Huang and Huo (2023) showed that traditional methods are insufficient for detecting tunneling activities and that it is necessary to analyze attributes such as traffic characteristics and packet sizes [9].

2.4 The Importance of Feature Engineering

Feature engineering is a critical step in transforming raw network traffic into meaningful data. Character entropy (Shannon entropy) and sliding window analysis are prominent metrics in the literature, particularly used to detect anomalies in encrypted or tunneled traffic. Beyond this, the frequency of requests sent and specific fingerprints are also important in distinguishing between bot and human behavior.

3. Methodology

This section details the hardware selection, cybersecurity isolation architecture, data transmission pipeline, and feature engineering processes of a hybrid analyze system built on the Raspberry Pi 4, a low-resource IoT device.

3.1 Hardware Selection and Raspberry Pi Configuration

The Raspberry Pi 4 Model B (4GB RAM) was chosen as the edge device for this project. The main reasons for this choice are its low power consumption, compact design, and its processing capacity to run resource-intensive services such as Cowrie, Honeytrap, and Suricata simultaneously without performance loss. To enhance the physical security of the device and prevent isolation bypass, Wi-Fi was disabled at the hardware level (rfkill block wifi), and communication was limited to a wired network only.

3.2. Network Architecture: Isolated Hybrid Monitoring Environment (Double NAT)

Protecting the main network while opening the system to real attackers is the most critical security step of the project. For this purpose, a Double NAT architecture has been established:

- ***Physical Setup:*** A TP-Link Router has been positioned between the internet service provider modem and the Pi, creating a new subnet (192.168.50.0/24) completely isolated from the main network (192.168.1.0/24).

- ***Firewall:*** Access Control rules have been defined on the router to allow the Pi to access the internet, but it is guaranteed that it cannot access any device on the main network (lateral movement restriction).
- ***DMZ Chain:*** A chained DMZ configuration has been implemented from the main modem to the Router and from the Router to the Pi to allow attackers to reach the Pi. Network Protection (QoS): To prevent the Pi from being used as a DDoS weapon in the event of a system compromise, a speed limit of 1 Mbps upload and 5 Mbps download (Quality of Service) has been implemented via the router.
- ***Static IP Service:*** Unless it's a large organization, internet service providers assign dynamic IP addresses to users to access the World Wide Web. Since these addresses don't always remain the same, attracting an attack would be difficult due to the random IP changes. Therefore, a static IP service was purchased, allowing the Raspberry Pi to access the internet with a single fixed IP address.

3.3 Honeypot Services and System Hardening

The biggest challenge in setting up honeypot services is that the services request the same ports and log writing conflicts occur. These challenges have been overcome with the following solutions:

- ***SSH Port Migration:*** To allow Cowrie to use port 22, the Pi's main management port was moved to 60022, and brute-force attempts on this port were prevented with Fail2Ban tool rules.
- ***Suricata (IDS):*** Configured to listen to all traffic, the "Checksum Offload" feature on the network card was disabled (ethtool) to prevent packet drops.
- ***Log Management (Logrotate):*** To prevent the disk from filling up quickly, log files were rotated daily and stored in compressed form.

3.4 Pipeline and Central Analysis Software

The project's data architecture is designed asynchronously to minimize resource usage on the edge device (Raspberry Pi) while ensuring instant, lossless, and secure data transfer to the central server (Host). This architecture consists of two main layers: Edge and Host.

3.4.1 Edge Layer Data Processing Logic

On the Raspberry Pi side, a Python-based pipeline has been designed to combine data from three separate honeypot services that generate logs in different formats (JSON, syslog, alert).

- ***Aggregator (Normalization Service):*** Log files from Cowrie (SSH), Honeytrap (HTTP/FTP), and Suricata (IDS) services are monitored in real-time. The Aggregator script reads Cowrie's detailed JSON output, Suricata's alert messages, and Honeytrap's event data, converting them into a standard JSON format containing common fields such as "timestamp, source IP, destination port, protocol, and action," as well as data specific to each service. This format consists of 20 fields.
- ***Communication and FIFO Structure:*** Processed logs are transferred via a Named Pipe (FIFO) to reduce dependencies between services and increase speed. This method allows data to flow directly from memory to the next service instead of to disk.
- ***Publisher (Secure Sending and Plan B):*** The Publisher script receives the normalized data from the FIFO. Just before sending the data over the network, it is written to the local disk as *edge_backup.jsonl.* This mechanism is designed as a basic Plan B, ensuring that data is securely

stored on the Pi even in the event of an internet outage, and that it resumes from where it left off when the connection is restored.

3.4.2 Communication Protocol: TLS Encrypted MQTT

For data transfer, MQTT, the standard protocol of the IoT ecosystem, has been preferred.

- ***Lightweight and Stability:*** Thanks to MQTT's low overhead structure, Pi's processor resources are preserved and network traffic is minimized.
- ***Security:*** Data is sent to the broker configured on HiveMQ Cloud using the TLS 1.2 encryption layer. This prevents third parties from monitoring attacker logs flowing over the internet.

3.4.3 Central Side (Host) and Data Recording

The central host acts as the "listener," receiving incoming data.

- ***Asynchronous Subscriber Module:*** A Python-based subscriber script running on the host side subscribes to MQTT topics. Each incoming data packet is instantly validated (schema validation), and faulty/corrupted packets are filtered out at this stage.
- ***SQLite Database Entry:*** The validated data is INSERTed into the SQLite database, ready for the feature engineering.

3.5 Feature Engineering

In this phase of the project, normalized data stored in the SQLite database has been transformed into numerical and categorical features that will provide input for statistical and behavioral analyses. This process involves generating "intelligent features" from raw log data that allow us to understand the attacker's intent, the tools used, and the complexity level of the attack.

Below, the key feature groups added to the system through the developed Python modules (feature_engineering.py) and their importance in cybersecurity analysis are explained in detail:

3.5.1 Statistical and Entropy-Based Features

- ***Payload and URI Entropy:*** Character diversity analysis was performed on DNS queries, HTTP URI paths, and transmitted data (payload). Entropy values exceeding the 4.5 threshold, which is low in normal text, were flagged as an anomaly (IoT Entropy Anomaly), indicating that the data was encrypted or tunneled using methods such as Base64.
- ***Payload Length:*** The length of requests was recorded as an attribute to detect data exfiltration attempts.

3.5.2 Behavioral and Temporal Characteristics

- ***Burst Analysis***: When the time difference between two events (time_diff) falls below 1 second, this is marked as a "Burst" (intense attack wave). This metric is a key indicator of attacks carried out using automated tools such as brute-force and port scanning.

- ***Session Management:*** Attacks were grouped into sessions based on IP address and time. A 300-second (5-minute) inactivity period was considered the start of a new session, and the total time the attacker spent on the system was calculated.

3.5.3 Semantic and Intent Analysis Features

Raw text data (commands, user agents, etc.) was categorized to digitize the attacker's "intent":

- ***Command Intent:*** The commands written by the attacker on the Cowrie honeypot were categorized into groups such as "Downloader" (malware download), "Recon" (reconnaissance), "Persistence" (maintaining system persistence), and "Execution" (execution).
- ***Attack Pattern Detection:*** Regular expressions (regex) were used to search for known attack signatures such as SQL Injection, XSS, Path Traversal, and Shellshock within the requests, and these were tagged as specific_attack_type.
- ***Device and Tool Identification:*** HTTP User-Agent information was analyzed to determine whether the attacker was using a browser, a Python script, or a browser tool like Nmap/Masscan (client_tool_type).

3.5.4 Threat Classification and Risk Scoring

All the obtained attributes were combined to perform a final security assessment:

- ***Suricata Alarm Categorization:*** Raw alarms generated by Suricata IDS were converted into more understandable threat classes (threat_class) such as "Malware Activity", "Brute Force" and "Command and Control".
- ***Cyber Kill Chain Prediction:*** The attacker's stage (Reconnaissance, Infiltration, Installation, C2) was estimated based on the extracted attributes.
- ***Dynamic Risk Score:*** The most complex output of the system, the risk_score, was calculated using the weighted average of the burst rate, entropy anomaly, critical command usage, and IDS alarms. Based on this score, attacks were labeled at four levels: "Low", "Medium", "High" and "Critical".

This comprehensive feature engineering process transformed the raw log stack into a high-accuracy dataset for the project's subsequent phase: machine learning and statistical analysis, visualization, and reporting.

4. Conclusion

In this study, an isolated and highly interactive honeypot infrastructure capable of monitoring cyberattacks and tunneling activities targeting the Internet of Things (IoT) ecosystem in real time was successfully established. The first phase of the project, data collection and processing, was completed in accordance with the defined engineering objectives.

The key gains achieved during the system's installation and operation are summarized below:

- ***Secure and Isolated Monitoring Infrastructure:*** Cowrie, Honeytrap, and Suricata services running on Raspberry Pi 4, thanks to "Double NAT" and DMZ architecture, are capable of collecting real attack traffic without compromising the main network.

- ***Stable Pipeline:*** With the MQTT protocol and a unique Python-based pipeline architecture, the instantaneous transfer of logs in different formats to the central server is ensured. Thanks to the local backup mechanism (Plan B), data loss in adverse situations such as internet outages is prevented.
- ***Advanced Feature Engineering:*** Raw log data; The data has been transformed into over 20 meaningful features, including character entropy, temporal "burst" analysis, and command intent. This process has transformed the data from a mere collection of records into an analyzable, high-accuracy dataset.

Consequently, at this stage of the project, all the necessary data infrastructure and feature sets for statistical analysis and machine learning models have been fully prepared. The resulting cleaned and normalized "Master Dataset" provides a scientific basis for tunneling detection and attack classification studies to be conducted in the second phase of the project. This work represents a significant step in creating a domestic IoT attack dataset and addressing the lack of real-world data in the literature.

5. Appendix

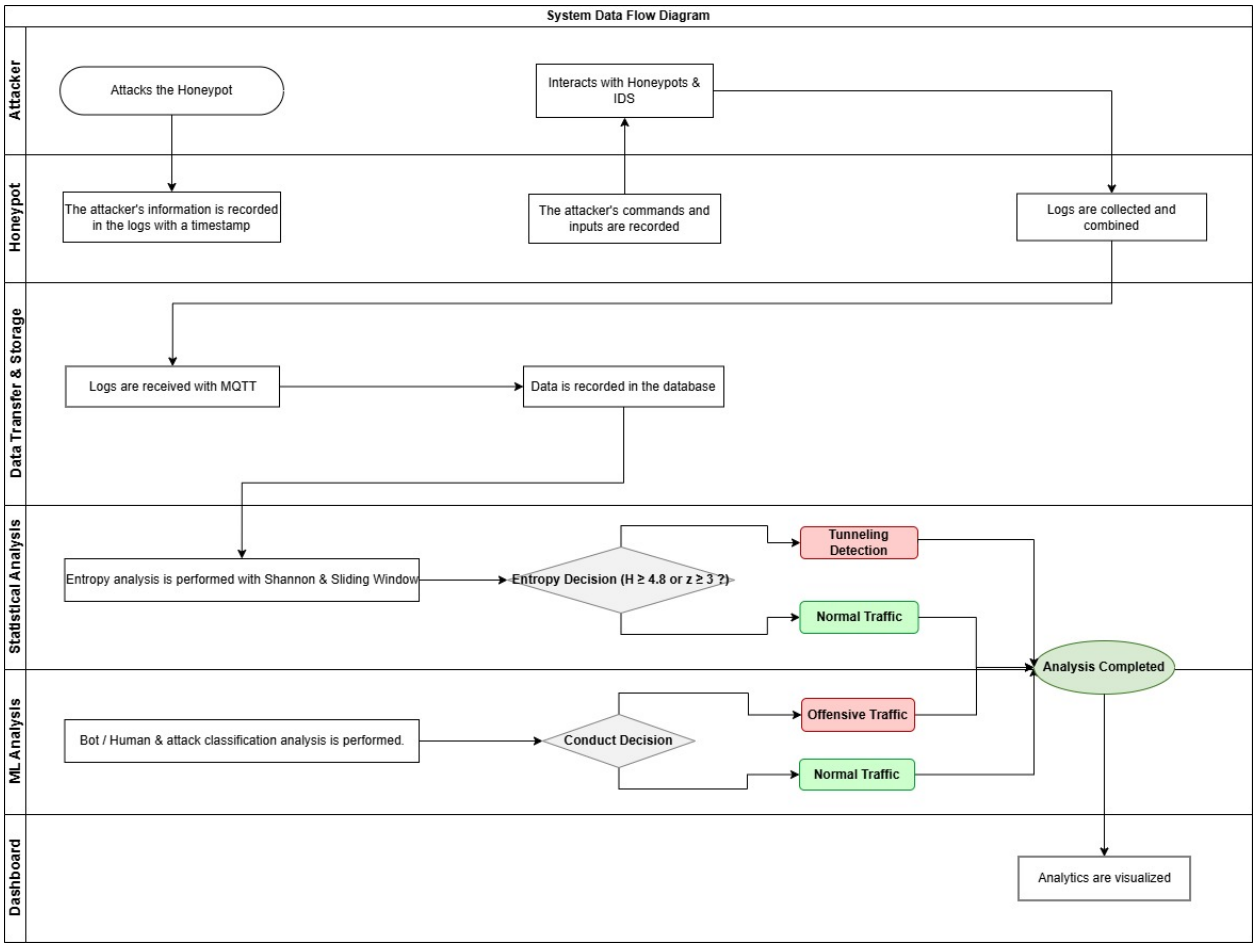

Figure 1 System Data Flow Diagram

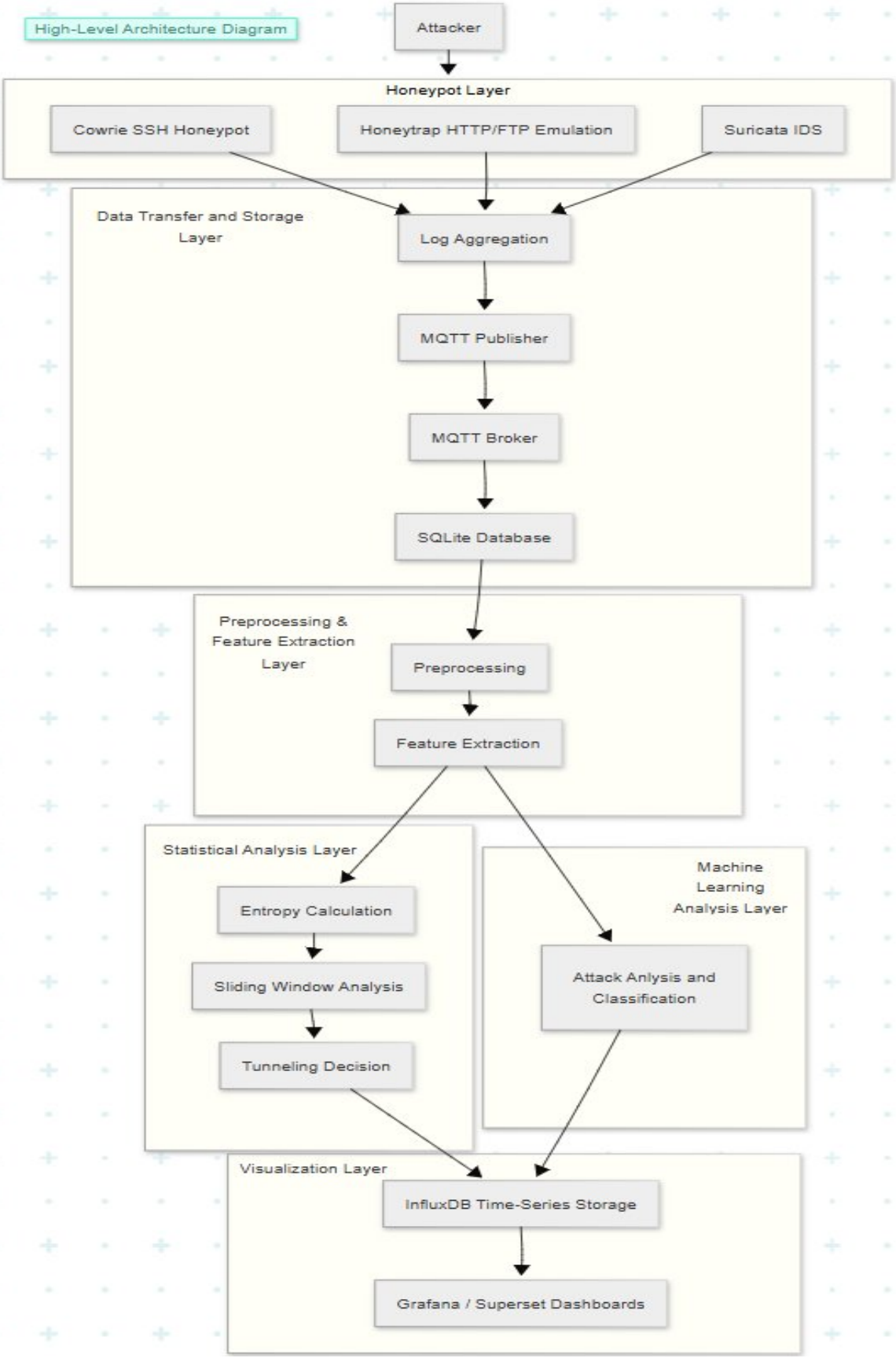

Figure 2 High Level Architecture Diagram

6. References

[1] M. Kaur and S. Singh, "IoT Network Security Threats and Countermeasures," Computers, vol. 14, no. 2, 2025.
Available: https://www.mdpi.com/2073-431X/14/2/61

[2] F. Al-Turjman, et al., "Federated Learning for IoT Security and Privacy," Future Internet, vol. 17, no. 1, 2025.
Available: https://www.mdpi.com/1999-5903/17/1/30

[3] R. N. Rao, "AI-Driven Detection of IoT Attacks using Honeypot Data," PubMed Central (PMC), 2024.
Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC10818527/

[4] A. Ahmed, et al., "Securing Smart Cities through Machine Learning: A Honeypot-Driven Approach to Attack Detection," IET Smart Cities, 2024.
Available:
https://www.openaccess.bcu.ac.uk/15567/1/IET%20Smart%20Cities%20-%202024%20-%20Ahmed%20-%20Securing%20smart%20cities%20through%20machine%20learning%20A%20honeypot%E2%80%90driven%20approach%20to%20attack-2.pdf.

[5] R. D. Venkat Goundla, "Analysis of IoT Honeypots for Network Security," Applied Sciences, vol. 15, no. 10, 2025.
Available: https://www.mdpi.com/2076-3417/15/10/5251

[6] V. G. Goundla, "Optimizing Internet of Things Honeypots with Machine Learning: A Real-World Study," Norma NCI Repository, 2024.
Available: https://norma.ncirl.ie/8173/1/venkatgoudgoundla.pdf

[7] X. Huang and Y. Huo, "A Comprehensive Review of Tunnel Detection on Multilayer Protocols: From Traditional to Machine Learning Approaches," Applied Sciences, vol. 13, no. 3, p. 1974, 2023.
Available: https://www.mdpi.com/2076-3417/13/3/1974

[8] Alve, "Smart IoT Security: Lightweight Machine Learning Techniques for Multi-Class Attack Detection in IoT Networks," arXiv preprint, arXiv:2502.04057, 2025.
Available: https://arxiv.org/abs/2502.04057

[9] A. Bhandari, T. Lyth, A. Shalaginov, and T.-M. Grønli, "Distributed Deep Neural-Network-Based Middleware for Cyber-Attacks Detection in the Smart IoT Ecosystem," Electronics, vol. 12, no. 2, 298, 2023.

[10] T. Salat, J. Davis, and M. Khan, "DNS Tunnelling, Exfiltration and Detection over Cloud Environments," Sensors, vol. 23, no. 5, 2760, 2023.

[11] F. Üstebay, "Enhancing Zero-Day Attack Detection in IoT Networks via Isolation," Electrica, vol. 25, no. 2, pp. 1–8, 2025.

[12] J. Mazel, M. Saudrais, and A. Hervieu, "ML-based Tunnel Detection and Tunneled Application Classification," arXiv preprint, arXiv:2201.10371, 2022.
Available: https://arxiv.org/abs/2201.10371

[13] "Intelligent Threat Detection—AI-Driven Analysis of Honeypot Data to Counter Cyber Threats," Electronics, vol. 13, no. 2465, 2024.
Available: https://www.mdpi.com/2079-9292/13/13/2465